Manual Test Scenarios — Import-Ready Saved Profiles

Purpose: step-by-step manual verification of the whole app against hand-computed (and API-verified) expected values. Each `.json` file is a saved-scenario profile in the client's import format (schema v3, `profile.service.ts`). Every expected number below was verified on 2026-07-21 against a live API instance built from commit `559aef2` (the current pushed state).

How to use

1. In the app, open **Saved Scenarios** → **Import** and pick one file. It is added as a new saved scenario (a fresh id is assigned; nothing is overwritten).
2. Load it (the restore icon), let the form populate, then **verify the key input fields below before pressing Calculate** — in particular *Calm Water Resistance Power*: profile restore and the vessel-catalog prefill can race; the field must show the scenario value (with the `(edited)` marker), not the catalog value. If it does not, load the scenario once more.
3. Calculate and compare against the **Expected** column. Fuel price is 780 USD/ton in all files — $ figures scale linearly if you change it.

All "Excel plant" scenarios (01–10, 12–13, 15–18) use: propulsion 11 463, hotel 3 800, SM 0 (16: SM 15), ME 2×24 000, SG 3 250/engine, AE 4×4 000, Transit 5 000 h — the scenario the reference workbook (`PowerPlantSetupAdvisesIncludingPTIOAndbatteries_test.xlsx`) was saved with, so Load Demands cells I/J/K/L (rows 8–9) and Optimal Setup Q7/R7/R8 can be cross-checked in Excel too.

> 📖 **Per-test calculation walkthroughs** live in calculations/ — one file per scenario, structured by result-panel section, showing the arithmetic behind every number on screen. Read them side-by-side with the loaded scenario.

Writing a new scenario file — the client's contract

A scenario `.json` is **a saved profile, not a request body**, and the importer validates it before the backend ever sees it. `ProfileService.isValidCalculatorInput` requires three fields the backend no longer reads at all — omit any of them and the file is refused with *"Invalid profile file: missing required fields"*, however valid it is server-side:

Field	Value to use	Why it is still required
`hotelLoad`	same as `transitHotelPowerKW`	legacy single-mode hotel load; kept equal so it can't be mistaken for an independent input
`batteryCapacity`	`0`	inert stub — the real battery is the `battery` object
`sailInstalled`	`false` (or the sail state)	superseded by `sailEnabled`

This is not a style rule: scenarios 19–35 were first written without them, every golden test passed, and all seventeen were un-importable in the UI. `KSailCalc.Tests/Golden/ScenarioImportContractTests` now mirrors the client's required-field list and fails the build instead. Write the files as 2-space JSON with **no BOM** (PowerShell's `ConvertTo-Json` / `Out-File` add one).

Scenarios and expected results

01 — Excel baseline (battery 1260 kW, Transit)

The workbook's saved state. Cascade: Propulsion I=573.15, J=**200.6025**, L=**372.5475**; Hotel I=76, J=**3.8**, L=**72.2**; remaining **610.85**.

- Tiles: **SR 444.7 / PS 204.4** · combos **5** · default baseline = **3rd-highest** (1 ME+SG+3 AE)
- Battery Benefit **173.66 t/yr ≈ $135 452/yr** · IL1 FOC **13 286.2 t/yr**
- Excel: J10=204.4025, L10=444.7475, R8=15 707.7475

Tier components (verified): IL1 savings **21.16 t/yr**; **IL2 adds 0** (L2 finds the SG+AE split already optimal) and **IL3 adds 0** on this plant (spike-cycle FOC delta clamps to zero at these load points, even though the panel shows variation 496.2 → 396.96 after the battery's 3.8 shave). All three tier chips therefore show the same 21.2 t/yr here — that is correct, not a bug.

02 — Small battery 300 kW (budget exhausts mid-cascade)

Propulsion grabs all 300 (J=**105**, L=**468.15**), Hotel gets **nothing** (L=76), remaining 0.

- Tiles: **SR 544.15 / PS 105** · Benefit **88.74 t/yr**
- Teaching point: PS+SR always = 649.15 (the swing is constant; the battery only moves the split).

03 — No-battery reference world (dual-scenario check)

No battery; loads pre-inflated to avg+full swing (propulsion 12 036.15, hotel 3 876) — this is the internal reference world the Benefit is computed against.

- IL1 optimal FOC **2.69198 t/h → 13 459.9 t/yr**
- Hand check: 13 459.9 − 13 286.2 (from 01) = **173.7 t/yr** = 01's Benefit line. R3a proven in the UI.

04 — DP redundancy 400 kW (RESERVE function)

DP mode 2 000 h, DP hotel 1 500, thrust 2 500, redundancy 400, battery 500 **DP-only**.

- Allocation table (DP): DpReserve **400/400/400/0** (covered 1:1, zero factor loss, first priority); Hotel 30/30/1.5/28.5; remaining 70
- Tiles: **SR 28.5 / PS 1.5** · Benefit **139.89 t/yr ≈ $109 113/yr**
- Note: Excel's J10 would show 401.5 (it sums the reserve row too); the PS tile counts peak-shaving rows only — not a mismatch.

05 — Mission crane 500 kW (cascade continues below)

Crane's variation = its FULL kW (starts at any moment). Mission row **500/500/250/250**, then Propulsion 573.15 (J 200.6025/L 372.5475), Hotel 76 (J 3.8/L 72.2), remaining **110.85**.

- Tiles: **SR 694.75 / PS 454.4** · Benefit **422.25 t/yr**

06 — Mission crane 3000 kW (budget devoured by priority)

Mission I=**1260** (whole budget), J=630, L=2370; Propulsion and Hotel get **zero**.

- Tiles: **SR 3019.15 / PS 630** · Benefit **631.29 t/yr**

- Bonus check (anti-double-counting, rule Q4/D4): the mission covered band (630, hotel side) fully absorbs the ±500 DRC variation → IL3 details show variation **0**, batteryShaved **500**, L3 component **0 t/yr**.

07 — Multi-mode Transit + Port

Each mode cascades with the FULL 1260 budget (modes never overlap in time). Two allocation tables: Transit = same as 01; Port = a single Hotel row **10/10/0.5/9.5** (no propulsion in port).

- Tiles (sums): **SR 454.25 / PS 204.9** · Benefit **173.74 t/yr** (Transit 173.66 + Port ~0.09)

08 — PTI 3250 (discharge gate passes)

Same results as 01 (SR 444.7 / PS 204.4, 5 combos) — headroom 6 500 ≥ band 200.6 → gate silent.

09 — PTI 50 (gate blocks: expect a 400 error, NOT results)

Expected red error: "*No feasible engine configuration: the battery needs 200.6 kW of PTI capacity to shave propulsion peaks in Transit mode, but only 100 kW is available. Increase the PTI capacity per main engine (currently 50 kW), reduce the battery power, or clear the PTI field…*" (QA-C-1). Threshold: 101/engine passes, 100/engine still fails (band 200.6025 vs 2×100+tolerance).

10 — Capacity plausibility warning (1000 kW / 400 kWh)

Warning: "*Battery capacity cannot sustain the configured power for 30 minutes…*" — a warning only; results still computed. Allocation identical to 01 (SR 444.7 / PS 204.4) because 1000 ≥ ΣH 649.15 — beyond saturation extra power changes nothing (INV-2).

11 — OSV 10 kn, all five modes (full pipeline flow)

The parametric Offshore Support vessel (curve 10 kn → 1500 kW, SM 15% → effective 1725).

- Battery tiles: **SR 60.24 / PS 30.41** (Prop H=86.25 → J 30.19/L 56.06; Hotel H=4.4 → J 0.22/L 4.18)
- Transit L1: only **2 combos**; optimal 1 ME+SG, ME 2 005.24 kW @ **13.37 %**, SFOC 174.78, FOC **0.35048 t/h**
- Per-mode FOC t/h: Transit 0.35048 · DP 0.66252 · Port 0.02627 · Anchor 0.03502 · Maneuv 0.13125
- Panels: Power Demands ME **2 012 kW / AE 0** · Baseline **3 081.6 t/yr** · IL1 **3 077.5** (savings only **4.1 t/yr**, negative ROI — flat SFOC curve + only 2 combos = nothing to optimize) · IL2 adds **0** (single generator carries hotel → nothing to redistribute) · IL3 chip **36.3 t/yr** = L1 4.1 + DRC component **32.2** (variation ±500 → −battery 0.22 → ×0.8 → **±400**, 30 cycles/h)

12 — Sail enabled (wind 10 m/s @ 90°, vessel 12.5 kn)

Sail thrust **539.93 kW** offsets propulsion: cascade Propulsion avg becomes **10 923.07** (11 463 − 539.93), H shrinks to **546.15**.

- Tiles: **SR 427.2 / PS 194.95** (smaller than 01 — the wind literally shrinks the swing)

Second wave — coverage of the remaining input families

13 — LNG main fuel v2 (per-fuel CO2 split, D6 fix #3)

v2 note: the first cut paired LNG with a Liquid-family engine — the client's fuel-family guard correctly coerced it back to MGO (and reset the price to the MGO default 950), so the LNG path never ran. v2 uses the **Dual Fuel**

Engine (id 5, 2×22 000, SG 2 800); LNG price default 620.

- Battery tiles **unchanged**: SR 444.7 / PS 204.4 (the cascade doesn't care about engines or fuel)
- Baseline **13 432.4 t/yr**, CO2 **38 410.7** = ME **33 639.6** (LNG 2.753) + AE **4 771.1** (MGO)
- IL1 **13 389.0 t/yr** (savings 43.4 t), CO2 **38 239.8** = ME **33 639.6** + AE **4 600.2** — the per-engine cards must show exactly these splits and sum to the panel totals (the pre-D6 single-constant bug cannot pass this with two different fuels).
- Note the reshuffled combo table (different SFOC curve): 2 ME+SG is now the SECOND row (2.6831), not the last; benefit 173.41 t/yr.
- After import verify: ME type = "Dual Fuel Engine", Main Fuel = LNG (now a legal option).

14 — Bulk Carrier, L3 variation from vessel-type lookup

Small bulk carrier (curve 12 kn → 1365 kW, SM 20 %); **Load Variation field left EMPTY** → backend looks up the vessel type: "Bulk Carrier…" → **±250**.

- IL3 Optimization Details: **Variation ±250 kW → ±200 kW** (the proof the lookup fired)
- Transit has only **1 valid combo** (SG 200 covers hotel 165 → AE idle) ⇒ baseline = optimal, IL1 savings **0**; L3 component also 0 at these load points. IL1 FOC = baseline = **1 883.36 t/yr**.
- After loading, verify the Load Variation field is empty; if the UI filled 250 as a visible default, that also confirms the type mapping.

15 — User-picked baseline (rule D1)

01 + `baselineIndex: 4` — pins the "Assumed Configuration" radio table: baseline forced to the **worst row (2 ME + SG, 0 AE)** instead of the battery default (3rd-highest).

- Baseline FOC **2.6975 t/h → 13 487.5 t/yr** · IL1 savings jump to **201.25 t/yr**
- The "Custom baseline" hint must appear under the table.

16 — Sea margin 15 % (the hidden battery link)

01 with SM 15: effective propulsion 11 463 × 1.15 = 13 182.45 → cascade H grows to 659.12.

- Tiles: **SR 500.63 / PS 234.49** · Benefit **199.67 t/yr** · Power Demands ME ≈ **16 861 kW**

17 — Infeasible plant (expect a 400 error, NOT results)

ME 2×5 000 (10 MW total) cannot carry 11 463 kW propulsion; no SG, no battery.

- Expected red error: "*Main engine utilization > 100%. Consider reducing propulsion power, decreasing sea margin, reduce hotel/mission load or increasing main engine capacity.*" (validation catches this before Level 1 even runs — different path than scenario 09).

18 — Battery assigned to Port, but Port hours = 0 (zero-effect guard, G2/B10)

Battery 1260 configured for Port only; Port hours 0.

- **No Battery Contribution panel at all** (backend returns no battery details), no crash, IL1 FOC = pure no-battery result **12 892.7 t/yr**. The battery silently does nothing — exactly as designed.

Coverage

See **COVERAGE-MATRIX.md** for what the 35 scenarios do and do not reach, verified against the approved snapshots rather than against the scenario files.

Scenarios 01–18 were verified against the reference workbook. **19–35 were generated from the current code** and are characterisation snapshots — they detect change, and each was checked to actually reach the path it targets, but figures marked "pending reference verification" in their cards are not yet correctness proofs.

Status of the logged findings (updated 2026-08-03)

#	Finding	Status
1	SG-forced rule runs an ME in Port/Anchor	**Open — needs a product decision** (changes the numbers)
2	DP weather factor never applied	**Open — needs a product decision** (missing feature + factor values)
3	DP redundancy value persists invisibly	**Open — low**; behaviour is harmless and the warning is correct
4	Fuel price read-only + two disagreeing price tables	**FIXED 2026-08-03**
5	`baselineIndex` lost on profile restore	**FIXED** (already closed before this pass, via `restoreInFlight` + source-tagged form events)

Finding 4 — how it was fixed

- The client's hard-coded `FUEL_DEFAULT_PRICES` table is **deleted**. It claimed to mirror the backend and had drifted on every fuel. The backend already ships the real prices in `AppInitialData.fuelDefaultPrices`, so both code paths now read `AppDataService.getFuelDefaultPrices()`. The 800 → 950 flicker is gone because there is no longer a second opinion about the price.
- `updateFuelPriceFromFuelType()` no longer overwrites unconditionally. It skips when `FormEditTrackerService.isFieldEdited('fuelPrice', …)` — so a price the user types now survives — and re-baselines the tracker whenever it does apply a default, so changing the fuel or engine still updates the price.

The product question the README previously raised ("should an edited price stick?") was answered by the code's own stated intent: `prefillPriceFromMainFuel` is documented as "the Main engine fuel drives the default fuel price **(editable afterwards)**". It was not editable.

Not covered by tests: the client has no test suite at all, so this fix is verified by `ng build` and by reasoning, not by an executed test. Worth a manual pass: type a fuel price, wait two seconds, confirm it stays; then change the fuel type and confirm the price follows.

Finding 7 — scenario 14's operating profile does not fit in a year (found & FIXED 2026-08-04)

`14-bulk-l3-lookup` carries **8935 h** of operating time: 5717 transit + 2592 port + 451 anchor + 175 maneuvering. A year has 8760. Nothing checked, so it was approved and re-approved unnoticed.

Every annual figure the calculator reports is a per-hour rate multiplied by these hours, so this scenario's fuel, CO2 and cost are overstated by ~2%.

Fixed on the backend, not in the scenario. `ValidationService` now emits an advisory `operating-hours` warning when the profile exceeds 8760 h — non-blocking, because a small overrun is usually rounding across modes and the user is better placed to judge. The scenario keeps its numbers and now carries the warning, which is the honest record.

Also fixed alongside it: `CalculatorInput.AnnualHours` counted **Transit + DP only**, so a vessel with port, anchor or maneuvering hours reported less than it operates. Nothing depended on the wrong value yet — its only reader was a `> 0` check already guaranteed by `transitHours` — but the next reader would have inherited the bug silently. It now sums every mode.

This is the ONLY approved snapshot that changed in the entire refactoring effort, and the diff is exactly one added warning: not a single number moved.

Finding 6 — the page calculates THREE times on load (found 2026-08-04)

Every page load fires `POST /calculate-all-variants` three times; afterwards it settles to one per edit. The backend does the full work all three times.

Mechanism. `emitFormValues()` is called from five uncoordinated places in `vessel-input-form.component.ts` — the debounced `valueChanges` subscription (151), the 1500 ms `scheduleInitialEmission` (224), the end of the vessel/engine cascade (596), `applyProfileInputValues` (684) and the weather handler (693). At startup three of them fire, because the cascade's steps are more than the 500 ms debounce apart, and a mid-cascade `patchValue(…, { emitEvent: true })` feeds one more event into the subscription. There is no single "the form is ready, calculate now" gate — there are five independent "I think I'm ready".

Why it looks fine. `calculationRequests$` uses `switchMap`, so only the last response is rendered. But `switchMap` cancels the client subscription, not the server's work.

Why it matters. Triple backend work per load; and it is the mechanism behind the already-logged symptom "an early fire can calculate with DEFAULT form values (fuel price 800) and leave mixed-price panels on screen".

Same root cause as findings 4 and 5. All three are an async cascade emitting more than once: the lost `baselineIndex` pin, the reverted fuel price, and now the triple calculation. Fixing the third symptom without collapsing the emissions into one gated stream will just move the problem.

Not fixed. The fix is a client refactor (five call sites → one Subject → one debounce → one calculation) and the client has **zero tests**. Write tests around profile restore and the vessel cascade first; otherwise this is a blind change to the code path that already produced three bugs.

Observations logged during scenario preparation (for review, not fixed)

1. **SG-forced rule** (`Level1OptimizationService.IsValid`, line ~183): when an SG is installed, every combination must run it — which forces an ME to run even in Port/Anchor (e.g. scenario 11: ME at 1 % load in port, AE fleet never used). Real vessels typically run AEs in port.
2. **DP weather factor is not applied anywhere**: `dpWeatherCondition` is stored and validated, but neither client nor backend multiplies `requiredDPPowerKW` by the profile's `thrustDemandFactor` (1.0/1.3/1.7). The dropdown is currently informational.
3. **DP redundancy value persists invisibly**: entering a redundancy value, then setting DP hours to 0, hides the field but keeps the value → harmless but produces the (correct) configuration warning "DP redundancy requirement is set but DP mode is not enabled".

4. **Fuel price is effectively read-only, and two price tables disagree** (found 2026-07-28 while reviewing the client):

- `updateFuelPriceFromFuelType()` runs on *every* debounced form change and overwrites `fuelPrice` with the main fuel's default whenever the two differ — so a user-typed price silently reverts ~500 ms later. The scenarios here never exposed it because their prices happen to equal the defaults (MDO 780, LNG 620).
- `shared/constants/fuel.constants.ts` claims to mirror the backend's `FuelDefaultPrices` but has drifted on **every** fuel: MGO 800 vs 950 · MDO 800 vs 780 · HFO 400 vs 420 · LNG 557 vs 620 · Ammonia 1100 vs 1350. The engine-selection path prefills from the stale client table, the form then resets to the backend value — a visible 800 → 950 flicker. Both need a product decision (should an edited price stick?), so nothing was changed.
